Describing a Robot Structure with Boxes

Overlapping-boxes algorithm on Blinky Blocks (VisibleSim)

Muhammad Omais Rana

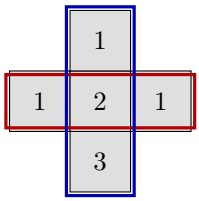
Abstract

A structure built from many small robots is hard to describe: listing every robot takes a lot of memory. We make the robots *work together* to describe the structure as a short list of rectangles (in 3D, boxes) that together cover every robot. No robot sees the whole structure and there is no central computer; robots only talk to the neighbours they touch. We implemented this in VisibleSim, tested it on five structures (with holes and in 3D) and compare it with the border-tracing algorithm.

1 The problem in one picture

Each robot is a small cube that knows only its own position and can talk to the robots touching it. Together they form a shape, possibly with holes.

Goal: describe the shape with as few "boxes" as possible, using only messages between neighbours.



Robots (grey squares)
Box A: the horizontal bar
Box B: the vertical bar

Numbers = "how many robots are above me in my column"

Figure 1: A plus-shaped structure of 5 robots is described by 2 boxes. (The algorithm also finds a tiny third box for the right-hand robot, but it lies inside Box A and is dropped.)

Instead of 5 positions we store 2 boxes, each given by two corners. For big structures the saving is large, and boxes can be *stacked* to describe 3D shapes.

2 How the robots find the boxes

The robots do it in four simple steps. Every step is just robots passing short messages to a neighbour.

1. **Count the height of each column.** The robot at the top of a column says "I am number 1" to the robot below, which says "then I am number 2", and so on. Each robot now knows how many robots are in its column above it. (The small numbers in the figure.)
2. **Pick the corners.** A robot at the bottom of a column asks: *is my left neighbour's column already the same height and ending at the same place?* If yes, that neighbour's box will already include me. If not, I become the **corner** of a new box.

3. **Measure the width.** A corner sends a message to the right: "I need columns at least as tall as mine, how far can I go?" The message passes through neighbours until a column is too short or the structure ends. The answer travels back, each robot adding 1, and the corner learns the box width. Height and width define the rectangle.

4. **Stack layers (3D).** Each robot tells the robot above it which rectangle it owns. If the rectangle above is *identical*, the two are really one taller box, so only the lowest one asks upward "how many identical layers are above me?" and learns the box height.

Robots run independently, so messages can arrive early. A robot that receives a question before it has finished its own counting simply keeps it and answers later.

Result. Every corner robot ends up knowing one box (two corners). Boxes may overlap; a box lying entirely inside another can be thrown away.

3 Tests

We tested five structures. All lie in a grid, with three layers or more in the 3D ones.

Structure	Description	Robots	Boxes	Messages	Time
`config`	2D shape with several holes	82	14	204	40.2 ms
`config_hole`	2D rectangle with a hole	44	4	96	20.1 ms
`config_3d`	4 stacked layers of config	216	17	698	47.3 ms
`config_hole_3d`	3D block with a hole through it	106	5	309	27.2 ms
`config_stair`	3D staircase of 5 steps	90	5	245	30.1 ms

Table 1: Results measured in VisibleSim. *Time* is the simulated time at which the last box was found (realtime scheduler, so it may vary slightly between runs); the message and box counts are exact and identical on every run.

What we observe.

- *Boxes cover everything.* After each run every robot is coloured, i.e. belongs to at least one box, and no box contains a hole.
- *Holes are no problem.* Boxes simply go around them (`config`, `config_hole`).
- *Identical layers merge.* In `config_3d` layers 0 and 1 are identical, so their boxes become boxes of height 2 instead of two separate ones.
- *The staircase needs one slab per step,* because every layer has a different size, so nothing can merge. That is the correct answer, not a failure.
- *Cost per robot is small and steady:* between 2.2 and 3.2 messages per robot in all five tests.
- *Time follows the size of the shape, not the number of robots.* The 216-robot `config_3d` took 47 ms, only about 2.3 times the 44-robot `config_hole` (20 ms), although it has almost five times as many robots.

2D structures

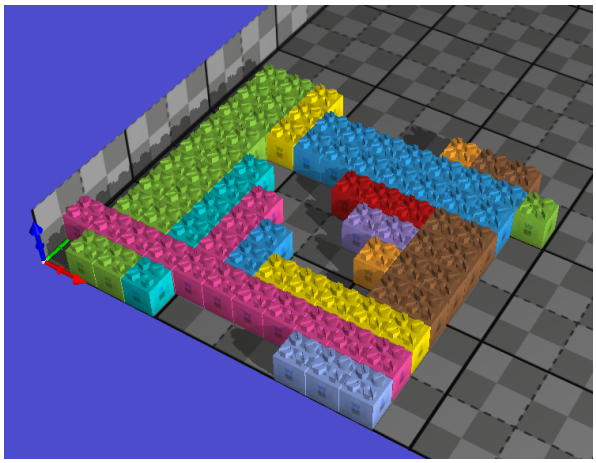


Figure 2: `config_1`: 14 boxes

3D structures

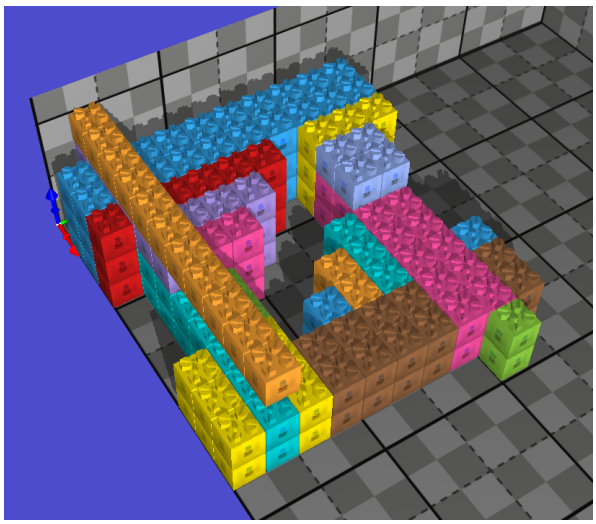


Figure 3: `config_3d`: 17 boxes

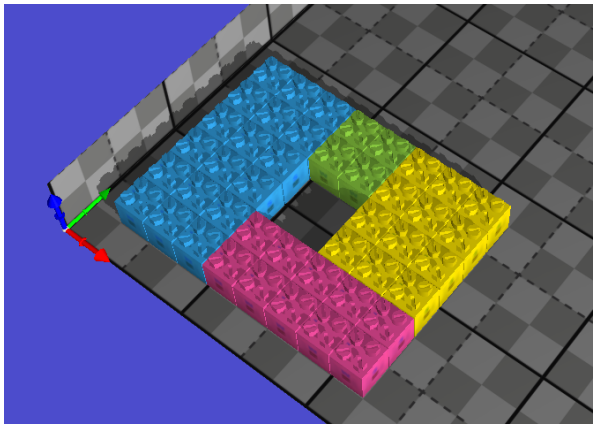


Figure 4: `config_hole`: 4 boxes

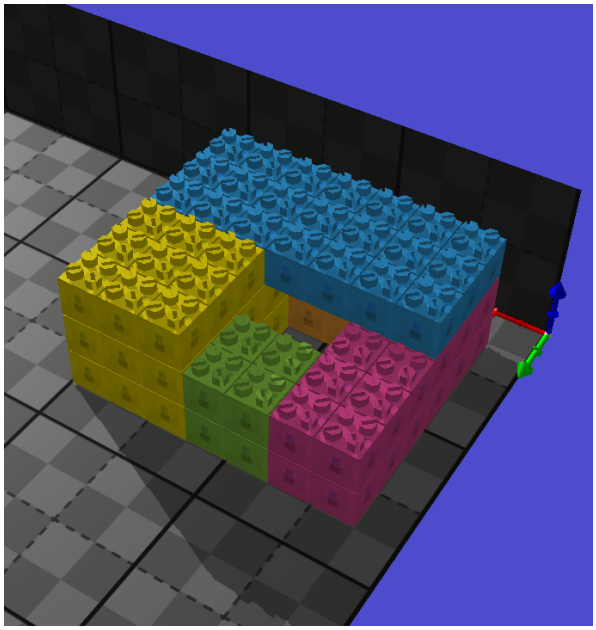


Figure 5: `config_hole_3d`: 5 boxes

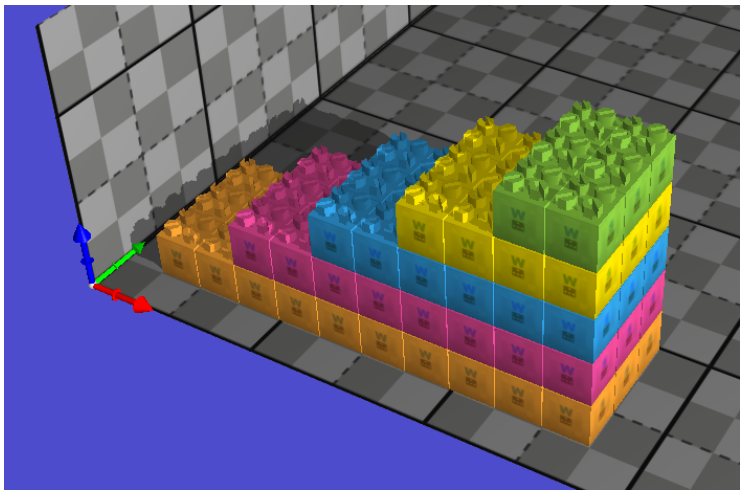


Figure 6: `config_stair`: 5 boxes

4 Complexity

We use these symbols: n = number of robots; L_x, L_y = longest row and longest column; H = number of layers (height); R = number of corner robots (at most one per column end); C = number of boxes found.

	Complexity	**In plain words**
Communication (messages)	$O(n)$ typical; worst case $O(n + R\,L_x + C\,H)$	Column counting, neighbour info and layer info cost at most n messages each. The width search costs at most $2L_x$ per corner and the height search at most $2H$ per box. In practice every robot sends only a few messages (2.2–3.2 per robot in our tests), so the total grows in proportion to n.
Time (parallel)	$O(L_x + L_y + H)$	Robots work at the same time. Counting columns takes as long as the longest column, measuring widths as long as the longest row, and finding heights as long as the tallest stack. A bigger structure is not slower unless it is also longer.
Memory per robot	$O(1)$ (worst case $O(L_x)$ briefly)	Each robot keeps a fixed handful of numbers. A robot may briefly hold width questions that arrive before it has finished counting; there are at most L_x of them and they are answered right away.
Memory of the result	$O(C)$	The structure is stored as C boxes of two corners each, instead of n positions.

Table 2: Complexity of the overlapping-boxes algorithm.

Evidence from our tests. The measured number of messages per robot stayed in a narrow band ($204/82 = 2.5$, $96/44 = 2.2$, $698/216 = 3.2$, $309/106 = 2.9$, $245/90 = 2.7$), which is consistent with the linear $O(n)$ behaviour. The simulated time grew much more slowly than the number of robots, as expected from $O(L_x + L_y + H)$.

5 Comparison with border tracing

Border tracing sends *one* message walking around the outline of the shape, robot after robot.

	Border tracing	**Overlapping boxes**
Idea	One message walks around the outline	Everyone measures its column, row and stack at the same time
Time	Slow: one robot at a time along the outline, $O(P)$	Fast: robots work in parallel, $O(L_x + L_y + H)$
Memory per robot	Small, but the message grows with the number of border segments	Small and fixed, $O(1)$
Messages	Few: only border robots take part, $O(P)$ for an outline of P robots	More: every robot takes part, $O(n)$
Holes	Each hole needs its own walk	Handled automatically
3D	Hard to extend	Natural: stack boxes upwards
Result	The outline	Full boxes covering the volume

In short: border tracing is cheaper in messages, overlapping boxes is faster and copes better with holes and 3D.

6 Conclusion

Robots that only talk to their neighbours can describe a whole structure as a short list of boxes, including structures with holes and in 3D. The cost is a few messages per robot and a time that follows the structure's length rather than its size. All five tests produced boxes that fully cover the structure.